QueueCare: Requirements and Technical Design Report

QueueCare is a MERN hospital token and queue system with three roles (Admin, Receptionist, Doctor), and the two specifications define 14 requirement areas that all persist to MongoDB. This report summarizes the scope, data design, queue rules, security, test plan and current build status. Only the Mongoose models from the first prototype exist today; the API, UI and tests are not built yet.

1. Background and problem statement

In many hospital outpatient departments, patients still queue on paper slips or shout-out systems. Staff cannot see how many people are waiting, patients do not know when their turn will come, and doctors have no reliable list of who is next. This causes long and uneven waiting times, duplicated or lost slips, queue-jumping disputes and no data for management to plan staffing.

QueueCare replaces the paper slip with a database-backed token. A receptionist registers the patient once, issues a numbered token for a department and doctor, and every later step (called, in consultation, completed) is recorded with a time. Administrators then have real figures for patients seen, waiting time and doctor load.

1.1 Objectives

- Issue unique, sequential tokens per department and day, even under simultaneous use.
- Give each role only the screens and actions it needs.
- Keep every business record in MongoDB, so data survives refreshes and restarts.
- Show live dashboard figures calculated from stored records.
- Keep the system simple enough to run locally from VS Code with two commands.

1.2 Out of scope

Billing, electronic medical records, appointment booking, SMS notification, payment and multi-hospital support are not part of this release. They are listed under future enhancements in section 18.

2. Scope and stack

Layer	Choice	Notes
Frontend	React + Vite, modern CSS, Lucide icons	Blue, white and teal healthcare theme; responsive on desktop, tablet, mobile
Backend	Node.js + Express	Separate controllers, routes, middleware; CORS limited to `CLIENT_URL`
Database	MongoDB + Mongoose, database `queuecare_db`	Local MongoDB or Atlas via `MONGODB_URI`
Auth	JWT + bcrypt, role-based middleware	Roles: Admin, Receptionist, Doctor

React never talks to MongoDB directly. No mock responses, no localStorage substitutes, no hardcoded dashboard numbers.

3. Requirements by module

Module	What it must do	Done when
Login	Email and password login against stored hashes; protected routes and APIs	Wrong password and wrong role are both rejected by the server
Dashboard	Total patients, today's tokens, waiting, in consultation, completed, active doctors, queue table, quick actions	Every figure comes from a MongoDB query and changes after a status update
Patients	Create, view, search, edit, deactivate; name, age, gender, phone	Invalid input is rejected with field-level errors
Doctors (Admin)	Add, view, edit, deactivate; specialization, department, availability	Searchable table; deactivated doctors cannot receive new tokens
Departments (Admin)	Create, view, edit, deactivate; unique names	Duplicate name returns a clear 409 error

Module	What it must do	Done when
Token generation	Select patient, department, doctor; sequential number per queue and date; success panel	Two simultaneous requests never produce the same number
Queue management	Statuses Waiting, Called, In Consultation, Completed, Cancelled, No-show; Call Next, Start, Complete	Status survives page refresh and server restart
Doctor dashboard	Doctor sees only their own queue and current patient	Another doctor's token action returns 403
Operations	`GET /api/health`, seed script, secure admin setup, README	Health endpoint reports database state without credentials

4. Data model

Six collections, all with timestamps and ObjectId references. Passwords are stored as bcrypt hashes and excluded from every API response.

Collection	Key fields	Indexes and rules
`users`	name, email, password hash, role, isActive	Unique email; `select: false` on password
`patients`	name, phone, age or date of birth, gender, isActive	Index on phone; text index on name
`doctors`	name, specialization, department ref, availability, isActive	Index on department + availability
`departments`	name, description, isActive	Unique name
`tokens`	token number, patient, doctor, department, queue date, status, timestamps	Unique on department + queue date + token number
`auditlogs`	user ref, action, affected record, timestamp, non-sensitive metadata	Index on user and timestamp

Records referenced by past tokens are deactivated, never deleted, so queue history stays intact.

5. Queue rules and concurrency

A token moves through a fixed set of states, and every change is written to MongoDB before the UI updates.

1. Waiting: created by the receptionist.
2. Called: set by Call Next, which picks the earliest Waiting token for that doctor or queue.
3. In Consultation: set by Start Consultation.
4. Completed: set by Complete Consultation.
5. Cancelled or No-show: set from Waiting or Called when the patient does not proceed.

Safety measures:

- Token numbers come from a counter document incremented with `$inc` and upsert, keyed by department and date, so concurrent requests cannot collide. The unique index is the second line of defence.
- Call Next uses a single `findOneAndUpdate` filtered on status Waiting, sorted by token number, so two clicks cannot call the same patient.
- A partial unique index allows at most one active token (Called or In Consultation) per doctor.
- Token creation first checks that the patient, doctor and department exist and are active. Transactions wrap steps that must succeed together (counter, token, audit log); they need a replica set, which Atlas provides and a local setup must enable.

6. API, security and setup

Access is enforced by backend middleware, never by hiding buttons in the UI.

Area	Endpoints	Allowed roles
Auth	login, current user	Public (login), any signed-in user
Patients	list, search, create, update, deactivate	Admin, Receptionist
Doctors and departments	list, create, update, deactivate	Admin (read: all signed-in roles)
Tokens	create, list, status transitions	Receptionist creates; Doctor changes only their own tokens
Dashboard	statistics	All signed-in roles

Area	Endpoints	Allowed roles
Health	`GET /api/health`	Public, no credentials in output

Security and setup rules:

- Server-side validation on every input, with clear errors for invalid ObjectIds, duplicates, missing records and database failures.
- `.env` is git-ignored; `.env.example` lists `PORT`, `MONGODB_URI`, `JWT_SECRET` and `CLIENT_URL` only.
- The first Admin is created by a one-time setup script that prompts for the password, so no credential lives in code.
- A seed script adds fictional departments, doctors and patients for demos.
- The server connects to MongoDB before it listens, and reports a failed connection instead of serving fake data.

7. Test and verification plan

Check	Expected result
Create, edit, search, deactivate a patient	Document appears, changes and flips `isActive` in `patients`
Add doctors and departments as Admin	Records saved; Receptionist attempting the same gets 403
Generate 20 tokens in parallel for one queue	20 distinct sequential numbers, no duplicates
Token for an inactive patient or doctor	Rejected with a clear error
Waiting to Called to In Consultation to Completed	Each step stored; dashboard counts update
Doctor acts on another doctor's token	403, no change in the database
Refresh browser and restart backend	Queue and statuses unchanged
Stop MongoDB	`/api/health` reports database down; no fallback data shown

8. Users and permissions

Three roles share one login page. The server decides what each role may do.

Capability	Admin	Receptionist	Doctor
Sign in and view own profile	Yes	Yes	Yes
Manage users (create, deactivate)	Yes	No	No
Manage departments	Yes	View only	View only
Manage doctors	Yes	View only	View own profile
Register and edit patients	Yes	Yes	View only
Deactivate patients	Yes	Yes	No
Generate tokens	Yes	Yes	No
View full queue	Yes	Yes	Own queue only
Call Next, Start, Complete	Yes	Call Next only	Own tokens only
Cancel or mark No-show	Yes	Yes	Own tokens only
View dashboard	Full	Full	Own statistics
View audit logs	Yes	No	No

A Doctor account is linked to one record in the `doctors` collection. The server compares the token's doctor with the signed-in doctor on every status change and answers 403 when they differ.

9. Detailed functional requirements

9.1 Authentication

1. The user enters email and password; the server finds the user, compares the bcrypt hash and checks `isActive`.
2. On success the server returns a JWT containing user id and role, with an expiry (for example 8 hours).
3. The client stores the token in memory and sends it in the `Authorization: Bearer` header on every request. Expired or invalid tokens return 401 and send the user back to login.
4. Passwords need at least 8 characters. Failed logins return one generic message so that the response does not reveal whether the email exists.

9.2 Patient management

- Fields: full name (required, 2 to 80 characters), age (0 to 120) or date of birth, gender (Male, Female, Other), phone (7 to 15 digits), creation date set by the server.
- Search matches name or phone, case-insensitive, with pagination (default 10 rows per page).
- Editing re-runs all validation. Deactivation hides the patient from new-token pickers but keeps old tokens readable.
- A duplicate phone number triggers a warning that lists the existing patient, because family members can share a phone.

9.3 Doctor and department management

- A department needs a unique name and may have a description. Deactivating a department is blocked while active doctors belong to it, or the doctors are moved first.
- A doctor needs a name, specialization, one department and an availability value (Available, On Break, Off Duty). Only Available and active doctors appear in the token form.
- Both tables support search, a status filter and inline edit dialogs.

9.4 Token generation

1. The receptionist picks a patient (search-as-you-type), a department, then a doctor filtered by that department.
2. The client posts the three ids. The server validates all three, takes the next counter value for the department and today's date, and creates the token with status Waiting.
3. The success panel shows token number, patient name and age, doctor, department, issue time and the number of patients ahead.
4. If anything fails (inactive doctor, database error), nothing is created and the counter does not skip silently without being logged.

9.5 Queue management

- The queue table lists token number, patient, department, doctor, registered time, live waiting time (now minus registration) and a coloured status badge.
- Filters: date (default today), department, doctor, status. Sorting defaults to token number.
- Row actions depend on status and role. For example, Complete only appears on In Consultation rows, and only to the doctor who owns the token.
- Every transition stores the new status, the time of change and the user who made it.

9.6 Doctor dashboard

The doctor sees a Current Patient card, the next three Waiting patients, counts for today (waiting, seen, remaining) and three buttons: Call Next, Start Consultation and Complete. Buttons that do not apply are disabled, and the server still rejects invalid transitions.

10. Non-functional requirements

Area	Requirement	How it is checked
Performance	Common list pages respond in under 500 ms with 10,000 tokens stored	Time the API with seeded data
Reliability	No data loss on restart; failed writes return an error, never a fake success	Restart test in section 7
Security	Hashed passwords, JWT expiry, role checks on every route, no secrets in the client	Review of middleware and `.env` handling
Usability	A receptionist can issue a token in under 30 seconds	Timed walkthrough
Responsiveness	Usable at 360, 768 and 1280 px widths	Browser device mode
Maintainability	Controllers, routes, models and middleware in separate files	Code review
Portability	Runs on Node 18 or newer with local MongoDB or Atlas	Fresh install on a clean machine

11. System architecture

The system has three tiers. The browser runs the React application. The Express server holds all business rules and is the only component that talks to MongoDB. MongoDB stores the six collections.

A typical request, such as Call Next, follows this path:

1. The doctor clicks Call Next; the React service layer sends `PATCH` with the JWT.
2. The authentication middleware verifies the token and loads the user.
3. The role middleware checks that the user is a Doctor or Admin.
4. The controller runs one atomic `findOneAndUpdate` on the earliest Waiting token for that doctor.

5. An audit log entry is written, and the updated token returns as JSON.
6. React updates the table and refreshes the dashboard statistics.

11.1 Folder layout

```
queuecare/
├── server/
|   ├── .env.example
|   ├── package.json
|   └── src/
|       ├── config/db.js
|       ├── models/        User, Patient, Doctor, Department, Token, Counter,
AuditLog
|       ├── controllers/   auth, patient, doctor, department, token,
dashboard
|       ├── routes/        one file per controller + health
|       ├── middleware/    auth, role, validate, errorHandler
|       ├── scripts/       seed.js, createAdmin.js
|       └── server.js
└── client/
    ├── package.json
    └── src/
        ├── api/           axios instance + service files
        ├── context/       AuthContext
        ├── components/    Sidebar, Topbar, Table, Modal, Badge, Toast
        └── pages/         Login, Dashboard, Patients, Doctors, Departments,
                           GenerateToken, Queue, DoctorDashboard
```

12. Database field reference

Collection	Field	Type	Rules
users	email	String	Required, unique, lowercase
users	passwordHash	String	Required, hidden from queries
users	role	String	Admin, Receptionist or Doctor
users	doctor	ObjectId	Required when role is Doctor
patients	name	String	Required, 2 to 80 characters
patients	phone	String	Required, 7 to 15 digits
patients	age	Number	0 to 120

Collection	Field	Type	Rules
patients	gender	String	Male, Female or Other
doctors	department	ObjectId	Required, references departments
doctors	availability	String	Available, On Break or Off Duty
departments	name	String	Required, unique
tokens	tokenNumber	Number	Required, at least 1
tokens	queueDate	String	Format YYYY-MM-DD
tokens	status	String	Six allowed values, default Waiting
tokens	calledAt, startedAt, completedAt	Date	Set by each transition
auditlogs	action	String	For example TOKEN_CREATED, STATUS_CHANGED
auditlogs	metadata	Object	Ids and statuses only, never passwords or tokens

13. REST API reference

Method	Path	Purpose
POST	`/api/auth/login`	Sign in and receive a JWT
GET	`/api/auth/me`	Current user
GET, POST	`/api/patients`	List with search and paging; create
GET, PUT, DELETE	`/api/patients/:id`	Read, update, deactivate
GET, POST	`/api/doctors`	List, create
PUT, DELETE	`/api/doctors/:id`	Update, deactivate
GET, POST	`/api/departments`	List, create
PUT, DELETE	`/api/departments/:id`	Update, deactivate
POST	`/api/tokens`	Generate a token
GET	`/api/tokens`	Queue list with filters
PATCH	`/api/tokens/call-next`	Atomically call the next waiting token

Method	Path	Purpose
PATCH	`/api/tokens/:id/start`	Start consultation
PATCH	`/api/tokens/:id/complete`	Complete consultation
PATCH	`/api/tokens/:id/cancel`	Cancel or mark No-show
GET	`/api/dashboard/stats`	Dashboard figures
GET	`/api/health`	Application and database status

13.1 Error responses

Status	Meaning	Example
400	Invalid input or ObjectId	Age of 250
401	Missing or expired token	Not signed in
403	Role or ownership not allowed	Doctor edits another doctor's token
404	Record not found	Unknown patient id
409	Duplicate or invalid state change	Department name exists; Complete on a Waiting token
500	Unexpected failure, logged on the server	Database write error
503	Database unavailable	Health check while MongoDB is down

14. User interface design

14.1 Visual style

The theme uses a deep blue for the sidebar and primary buttons, teal for accents and success states, white cards on a light grey page, and rounded corners with soft shadows. Text is dark slate for contrast. Status badges use colour plus a text label so that colour is never the only signal.

Status	Badge colour	Meaning for staff
Waiting	Amber	In line, not yet called
Called	Blue	Asked to go to the room

Status	Badge colour	Meaning for staff
In Consultation	Teal	With the doctor now
Completed	Green	Finished
Cancelled	Grey	Withdrawn before consultation
No-show	Red	Did not respond when called

14.2 Screens

Screen	Main elements
Login	Centered card, logo, email, password, show-password toggle, error message area
Dashboard	Six summary cards, queue table for today, quick-action buttons (New Patient, Generate Token, Open Queue)
Patients	Search box, table, Add button, edit and deactivate actions, confirmation dialog
Doctors	Searchable table, department filter, add and edit dialogs, availability dropdown
Departments	Table with active status, add and edit dialogs
Generate Token	Three-step form and a success panel with a Print button
Queue	Filters, live waiting time, action buttons by status
Doctor Dashboard	Current patient card, next patients list, three action buttons

14.3 Interaction rules

- Every request shows a loading indicator and disables its button to stop double submission.
- Destructive actions (deactivate, cancel) ask for confirmation.
- Success and error messages appear as toast notifications for about four seconds.
- Empty tables explain what to do next, for example "No patients yet. Add the first patient."

- On phones the sidebar collapses into a menu button and tables scroll sideways inside their own container.

15. Seed data and administrator setup

The seed script creates fictional records only and refuses to run if `NODE_ENV` is `production` unless a flag is passed.

Data	Sample content
Departments	General Medicine, Cardiology, ENT, Dermatology, Orthopedics
Doctors	Two per department with fictional names and specializations
Patients	About 20 fictional patients with varied ages and genders
Users	One Receptionist and one Doctor account per seeded doctor, with passwords generated randomly and printed once in the terminal

The administrator is created with a separate command that asks for name, email and password at the prompt and stores only the hash. No default password appears in the repository, README or code.

16. Risks and mitigations

Risk	Likelihood	Impact	Mitigation
Two receptionists get the same token number	Medium	High	Atomic counter plus unique index
Two clicks of Call Next call the same patient	Medium	High	Single atomic update with status filter
Transactions fail on a standalone local MongoDB	High	Medium	Document replica-set setup, or use Atlas
Secrets committed to GitHub	Medium	High	`.gitignore` for `.env`, `.env.example` with placeholders
Stolen JWT used after logout	Low	Medium	Short expiry, HTTPS in production

Risk	Likelihood	Impact	Mitigation
Doctor sees other doctors' patients	Low	High	Ownership check in the backend on every action
Scope grows beyond the schedule	High	Medium	Core workflow first; extras only after tests pass

17. Delivery plan

Phase	Work	Output
1	Models, database connection, health endpoint	Server starts and connects to `queuecare_db`
2	Auth and role middleware	Login works; protected routes return 401 or 403
3	Patients, doctors, departments APIs	CRUD verified in MongoDB
4	Token generation and queue transitions	Concurrency test passes
5	Dashboard statistics and audit logs	Figures match the database
6	React pages and styling	End-to-end workflow in the browser
7	Seed, admin script, README, final tests	Fresh-machine setup works

18. Future enhancements

- Live updates through Socket.io and a public waiting-room display.
- SMS or WhatsApp alerts when a token is two places from being called.
- Appointment booking with time slots alongside walk-in tokens.
- Reports: average waiting time, patients per doctor, peak hours, with CSV export.
- Priority tokens for emergencies and senior citizens.
- Multi-branch support and a refresh-token flow for longer sessions.

19. Gaps against what is already delivered

The earlier prototype models differ from the new specification and need rework before the API is built.

Item	Delivered now	Specification requires
Doctors	Stored inside the `User` model	Separate `doctors` collection linked to a user account
Token statuses	Waiting, Serving, Completed, Skipped	Waiting, Called, In Consultation, Completed, Cancelled, No-show
Audit trail	Not present	`auditlogs` collection
Departments	Has a code and colour, no description	Needs a description field
Patients	No active flag	Needs `isActive` for deactivation
Real-time TV screen and Socket.io	Planned in the first brief	Not mentioned in either specification
Roles	Includes Admin, Receptionist, Doctor	Matches

The token counter, daily numbering and unique token indexes carry over unchanged.

20. Status and next steps

Nothing has been run or tested yet, and no feature in the specification is verified as working. Suggested order of work:

1. Align the models with section 19 and add `AuditLog`.
2. Build the database connection, auth, role middleware and health endpoint.
3. Build patient, doctor and department CRUD, then token generation and queue transitions.
4. Build the React pages, then the seed and admin-setup scripts and the README.
5. Run the section 7 checks and record the results.

Open question: keep Socket.io live updates as an optional extra, or leave them out to prioritize core reliability as the second specification asks?

21. Conclusion

QueueCare is a focused system: a receptionist registers a patient and issues a token, a doctor works through the queue, and every step is saved in MongoDB. The design puts correctness first, using atomic updates, unique indexes, server-side role checks and

deactivation instead of deletion. The first priority is to align the existing models with the new specification, then build in the phase order of section 17 and prove each requirement with the checks in section 7.

22. Glossary

Term	Meaning
Token	A numbered queue ticket issued to a patient for one doctor visit
Queue date	The calendar day for which the token number sequence applies
Atomic update	A single database operation that cannot be interrupted or half-applied
JWT	JSON Web Token, a signed proof of sign-in sent with each request
RBAC	Role-based access control, permissions decided by the user's role
Deactivation	Marking a record inactive instead of deleting it, so history stays intact
Audit log	A record of who did what and when